LAPORAN PRAKTIKUM MIKROPROSESOR DAN INTERFACE

OPERASI PERKALIAN BILANGAN HEKSADESIMAL

Disusun untuk memenuhi tugas praktikum Mikroprosesor Dan Interface

Oleh

Gaitsha Azzahra Devitrya

231344012

Tanggal Percobaan: 27 Maret 2025

Tanggal Penyerahan: 25 April 2025

Instruktur: 1. Ferry Satria, BSEE., M.T.

2. Rahmawati Hasanah, S.ST., M.T.

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2025

DAFTAR ISI

DAFTAR ISI.....ii
A. TUJUAN.....2
B. LANDASAN TEORI.....2
C. ALAT DAN BAHAN.....4
D. LANGKAH KERJA.....4
E. HASIL PERCOBAAN DAN ANALISIS DATA.....5
G. KESIMPULAN.....17
DAFTAR PUSTAKA.....18

A. TUJUAN

1. Mahasiswa dapat memahami konsep operasi perkalian bilangan heksadesimal.
2. Mahasiswa dapat membuat program pada EMU8086 untuk menyelesaikan operasi perkalian bilangan heksadesimal.

B. LANDASAN TEORI

A. Definisi microprosesor

Mikroprosesor dikenal juga sebagai CPU (Central Processing Unit). CPU terdiri dari ALU, Control Unit, Register , Bus System, dan Clock untuk melakukan komputasi.

B. Register

Register merupakan sebagian memori dari mikroprosesor yang dapat diakses dengan kecepatan yang sangat tinggi. Dalam melakukan pekerjaannya mikroprosesor selalu menggunakan register-register sebagai perantaranya, jadi register dapat diibaratkan sebagai kaki dan tangannya mikroprosesor.

Register merupakan sebagian memori dari mikroprosesor yang dapat diakses dengan kecepatan yang sangat tinggi. Dalam melakukan pekerjaannya mikroprosesor selalu menggunakan register-register sebagai perantaranya, jadi register dapat diibaratkan sebagai kaki dan tangannya mikroprosesor.

Terdapat dua jenis register yang dibedakan menurut besar Bit, yaitu Register 8-Bit dan Register 16-Bit.

Register 8-Bit hanya memiliki 1 bagian jenis register yaitu Register umum : AH, AL, BH, BL, CH, CL, DH, DL

Dan Register 16-Bit memiliki 5 bagian jenis register yang memiliki tugasnya masing-masing, yaitu :

1. Segmen Register.

   Register yang termasuk dalam kelompok ini terdiri atas register CS (Code Segment), DS (Data Segment), ES (Extra Segment) dan SS (Stack Segment) yang masing-masingnya merupakan register 16 bit. Register-register dalam kelompok ini secara umum digunakan untuk menunjukkan alamat dari suatu segmen.

2. General Register (Register Umum)

   Register yang termasuk dalam kelompok ini adalah register AX, BX, CX dan DX yang masing-masing terdiri atas 16 bit. Register- register 16 bit dari kelompok ini mempunyai suatu ciri khas, yaitu dapat dipisah menjadi 2 bagian dimana masing-masing bagian terdiri atas 8 bit, Akhiran H menunjukkan High sedangkan akhiran L menunjukkan Low.

3. Register Pointer

   Register ini menunjukkan alamat sebuah data di lokasi memori, dipakai saat operasi perpindahan data(dari/ke memori) dan penunjukkan alamat suatu instruksi. Jenis-jenis register pointer yaitu BP (Base Pointer) dan IP (Instruction Pointer).

4. Register Index

Sama dengan pointer register, sering digunakan untuk menunjukkan alamat sebuah data di lokasi memori pada operasi string. Jenisnya ada SI (Source Index), DI (Destination Index).

5. Register Stack

Register Stack (SP) atau Stack Pointer adalah salah satu jenis register dalam arsitektur komputer yang digunakan untuk menunjuk alamat teratas dari stack dalam memori.

6. Register Flag [REG 16 BIT]

Register Flag berfungsi untuk menunjukkan status(keadaan) sesaat dari mikroprosesor. Bit-bit pada flag akan mengalami perubahan, tergantung proses yang berlangsung. Sesuai dengan namanya Flags(Bendera) register ini menunjukkan kondisi dari suatu keadaan< ya atau tidak >. Karena setiap keadaan dapat digunakan 1 bit saja, maka sesuai dengan jumlah bitnya, Flags register ini mampu memcatat sampai 16 keadaan.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-	-	-	-	OF	DF	IF	TF	SF	ZF	-	AF	-	PF	-	CF

C. Bahasa Assembly

Bahasa Assembly (rakitan) adalah bahasa pemrograman tingkat rendah yang digunakan pada perangkat yang dapat di program seperti komputer dan mikroprosesor. Instruksi-instruksi pada bahasa assembly menggantikan kode-kode biner dari bahasa mesin dengan mnemonic yang mudah diingat. Misal, instruksi penambahan dalam bahasa mesin memiliki kode “10110011” sedangkan dalam bahasa assembly dapat dibuat dalam instruksi mnemonic ADD, sehingga lebih mudah diapahami oleh manusia.

Bahasa assembly mempunyai hubungan 1-1 dengan berdasarkan instruksi mesin. Contohnya, untuk setiap julukan atau mnemonic yang nanti akan ditulis dengan menggunakan bahasa assembly, maka nanti juga akan langsung diterjemahkan dengan tepat satu kode operasi sehingga bisa langsung dimengerti oleh komputer.

D. EMU8086

EMU8086 adalah emulator mikroprosesor 8086 yang dapat digunakan untuk mengemulasikan perangkat keras pada komputer Windows. Emulator ini memungkinkan pengguna untuk menjalankan program yang dibuat untuk mikroprosesor 8086 pada komputer modern.

EMU8086 juga dilengkapi dengan assembler terintegrasi dan tutorial untuk pemula. Emulator ini dapat digunakan untuk mengeksplorasi dan mempelajari

bahasa pemrograman assembly serta untuk mengembangkan dan menguji kode assembly pada mikroprosesor 8086.

E. FUNGSI EMU8086

EMU8086 memiliki beragam kegunaan dalam pembelajaran bahasa assembly, antara lain:

- Emulator 8086: Emu8086 dapat digunakan untuk menjalankan dan menguji program yang ditulis dalam bahasa assembly untuk mikroprosesor 8086. Hal ini memungkinkan para pengguna untuk mensimulasikan program kontrol hardware yang dibuat, serta dilengkapi dengan assembler terintegrasi dan tutorial untuk pemula
- Pembelajaran Bahasa Assembly: Emu8086 memungkinkan pengguna untuk mempelajari bahasa pemrograman assembly dan mikroprosesor 8086. Dengan menggunakan emulator ini, para pengguna dapat menjalankan, menguji, dan memahami cara kerja perangkat keras komputer serta mengembangkan kode assembly untuk mikroprosesor 8086
- Pengembangan Kode Assembly: Emu8086 dapat digunakan untuk mengembangkan dan menguji kode assembly pada mikroprosesor 8086. Hal ini memungkinkan para pengguna untuk membuat (compile) "binary executable file" dari kode program serta menjalankan program yang dibuat untuk mikroprosesor 8086 pada komputer modern

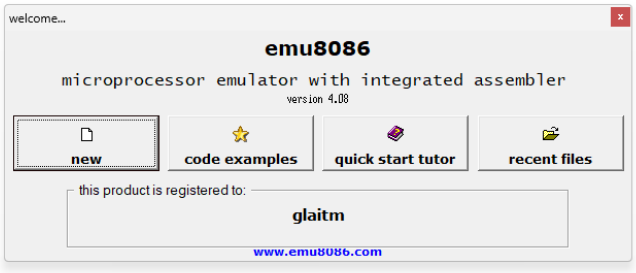
Dengan beragam kegunaan tersebut, EMU8086 merupakan aplikasi yang cocok digunakan untuk belajar bahasa assembly dan mikrokontroler serta untuk mengembangkan dan menguji kode assembly pada mikroprosesor 8086.

C. ALAT DAN BAHAN

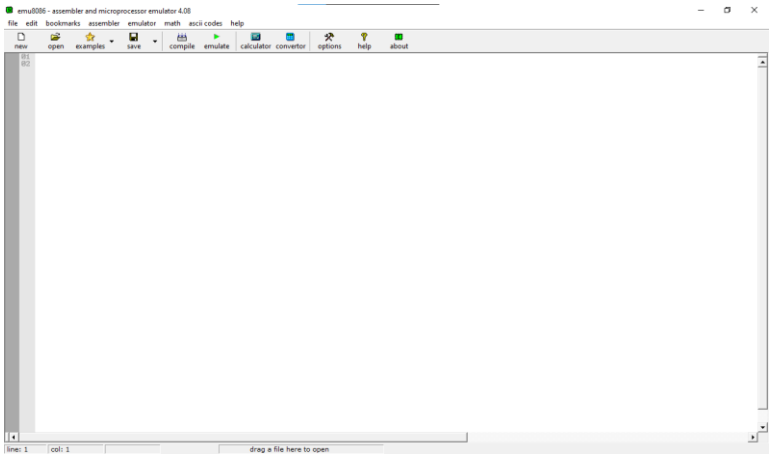
1. Laptop/PC
2. Software EMU 8086

D. LANGKAH KERJA

1. Instal Software emu8086 pada laptop dan buka aplikasinya
2. Setelah membuka aplikasi maka akan muncul tampilan *new* pada monitor

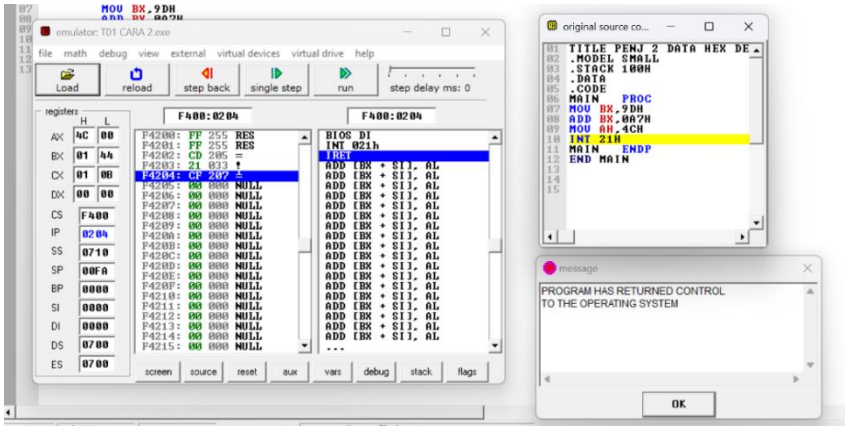


3. Maka setelah itu akan tampil worksheet seperti gambar dibawah ini :



Worksheet ini digunakan untuk menuliskan kode.

4. Untuk memeriksa kode yang berada di worksheet apakah program sudah benar atau belum, klik “compile”
5. Jika sudah disave maka jika program sudah benar akan ada tulisan “run”.
6. Jika sudah di ”run” maka akan muncul hasilnya pada monitor



E. HASIL PERCOBAAN DAN ANALISIS DATA

Pada praktikum ini, menggunakan aplikasi EMU8086 dimana praktikum ini mempelajari bagaimana cara membuat program untuk melakukan operasi perkalian bilangan heksadesimal pada bahasa assembly.

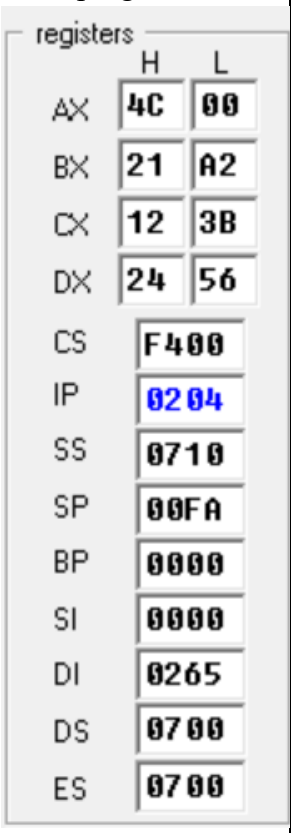
Pada praktikum kali ini terdapat 4 soal yang berhasil dikerjakan yang dapat dianalisis sebagai berikut :

1. Buat program untuk mengalikan 2 data heksadesimal 4 digit yang ada pada register BX dan CX.Simpan hasil kali pada register DI:DX.Gunakan 4 metoda.

1. Penjumlahan berulang

```
01 TITLE SOAL 1 METODE ;PENJ BERULANG
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 .CODE
06 MAIN PROC
07     MOV BX, 21A2H
08     MOV CX, 123BH
09
10     MOV AX, CX
11     MOV DX,0
12     MOV DI,0
13
14 LUP:ADD DX, BX
15     ADC DI, 0
16     DEC AX
17     JNZ LUP
18
19     MOV AH,4CH
20     INT 21H
21 MAIN ENDP
22 END MAIN
```

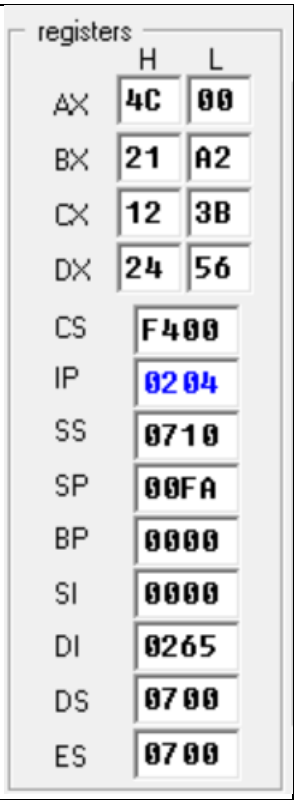
Hasil program



2. Instruksi MUL

```
01 TITLE SOAL 1 METODE ;MUL
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 .CODE
06 MAIN PROC
07     MOV BX, 21A2H
08     MOV CX, 123BH
09
10     MOV AX,BX
11     MUL CX
12
13     MOV DI,AX
14
15     MOV AH,4CH
16     INT 21H
17 MAIN ENDP
18 END MAIN
```

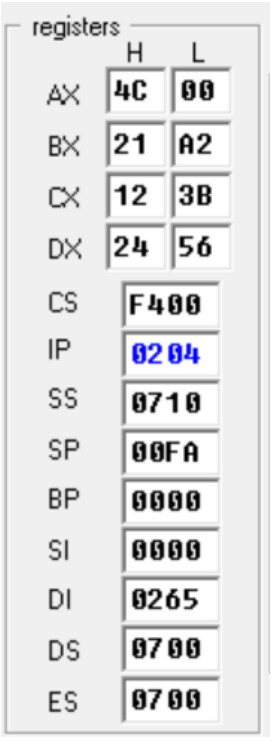
Hasil program



3. Pergeseran SHR

```
01  TITLE SOAL 1 METODE ;SHR
02  .MODEL SMALL
03  .STACK 100H
04  .DATA
05  .CODE
06  MAIN PROC
07      MOV AX,16
08
09  LUP:SHR CX,1
10      JNC LUP1
11      ADD DI,BX
12      ADC DX,0
13
14  LUP1:SHR DI,1
15      RCR DX,1
16      DEC AX
17      JNZ LUP
18
19      MOV AH,4CH
20      INT 21H
21  MAIN ENDP
22  END MAIN
```

Hasil program



4. Pergeseran SHL
`01 TITLE SOAL 1 METODE ;SHL`
`02 .MODEL SMALL`
`03 .STACK 100H`
`04 .DATA`
`05 .CODE`
`06 MAIN PROC`
`07 MOV AX,16`
`08 LUP:SHL SI,1`
`09 SHL DX,1`
`10 ADC SI,0`
`11 SHL CX,1`
`12 JNC LUP1`
`13 ADD DX,BX`
`14 ADC SI,0`
`15 LUP1:DEC AX`
`16 JNZ LUP`
`17`
`18 MOV AH,4CH`
`19 INT 21H`
`20 MAIN ENDP`
`21 END MAIN`
	Hasil program
	registers
	H L
	AX 4C 00
	BX 21 A2
	CX 12 3B
	DX 24 56
	CS F400
	IP 0204
	SS 0710
	SP 00FA
	BP 0000
	SI 0000
	DI 0265
	DS 0700
	ES 0700

Analisis data :

1) Pada program 1 dengan metode penjumlahan berulang dapat dianalisis, bahwa program merupakan implementasi algoritma perkalian dua bilangan heksadesimal menggunakan metode penambahan berulang. Pada awal program, dua bilangan heksadesimal, yaitu 21A2H dan 123BH, dimasukkan ke dalam register BX dan CX. Register AX digunakan sebagai counter untuk menentukan jumlah pengulangan perkalian, sementara DX dan DI digunakan untuk menyimpan hasil perkalian secara terpisah (Low dan High). Setiap iterasi, nilai BX ditambahkan ke DX, dan carry dihitung dengan instruksi ADC untuk ditambahkan ke DI. Proses ini akan diulang sebanyak nilai yang ada di register AX hingga menjadi 0, dan akhirnya program mengakhiri eksekusi dengan instruksi INT 21H.
2) Pada program 2 dengan metode instruksi MUL/multiply. Pertama adalah menginisialisasi segmen data menggunakan register AX untuk memuat alamat data, kemudian register DS di-set untuk menunjuk ke segmen data. Bilangan pertama dan kedua dimasukkan ke dalam register BX dan CX. Instruksi MUL digunakan untuk mengalikan nilai yang ada di AX dengan nilai yang ada di CX, sehingga menghasilkan hasil perkalian 32-bit yang disimpan di DX:AX. Hasil perkalian kemudian dipindahkan ke register DI untuk menyimpan nilai rendah, sementara nilai tinggi berada di DX.
3) Pada program 3 dengan metode pergeseran/shift right(SHR). Pada bagian awal, register AX diinisialisasi dengan nilai 16, yang menentukan jumlah bit perulangan yang dilakukan. Dengan menggunakan instruksi SHR pada

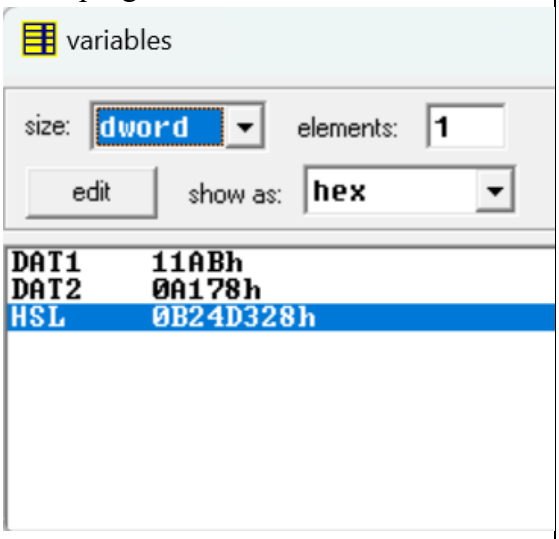
register CX, bit kanan akan pindah, yang setara dengan pembagian dengan 2 dalam setiap langkah. Dalam setiap iterasi, jika bit yang dipindahkan menyebabkan carry, nilai pada register BX akan ditambahkan ke register DI dengan instruksi ADD dan ADC. Selanjutnya, hasil dari pergeseran bit juga diproses dengan menggunakan instruksi RCR pada DX untuk mengatur carry bit. Proses ini diulang hingga AX mencapai 0, menandakan perhitungan selesai dan akhirnya program mengakhiri eksekusi dengan instruksi INT 21H.

4) Pada program 4 dengam metode pergeseran/shift left(SHL). Register AX di-set ke 16 untuk menghitung jumlah bit yang akan digeser. Lalu, DX menyimpan hasil bagian bawah, dan DI menyimpan hasil bagian atas. Kemudian, instruksi SHL dan ADC digunakan untuk melakukan pergeseran ke kiri dan penambahan carry. Setiap kali bit paling kiri dari CX adalah 1(setelah shift), maka BX ditambahkan ke DX, dan hasil carry-nya disimpan ke DI. Setelah 16 kali iterasi, hasil akhir dari perkalian akan disimpan di register DI:DX.

2. Sama seperti soal no.1 namun semua data input dan output disimpan pada data area. Gunakan 4 metoda.

1. Penjumlahan berulang	Hasil program

```
01 TITLE SOAL 2 METODE ;PENJ BE
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05      DAT1 DW 11ABH
06      DAT2 DW 0A178H
07      HSL  DD 0
08 .CODE
09 MAIN PROC
10      MOV AX,@DATA
11      MOV DS,AX
12
13      MOV BX,DAT1
14      MOV CX,DAT2
15
16      MOV AX,CX
17      XOR DX,DX
18      XOR DI,DI
19
20 LUP1:ADD DX,BX
21       ADC DI,0
22       DEC AX
23       JNZ LUP1
24
25       MOV WORD PTR HSL,DX
26       MOV WORD PTR HSL+2,DI
27
28      MOV AH,4CH
29      INT 21H
30 MAIN ENDP
31 END MAIN
```



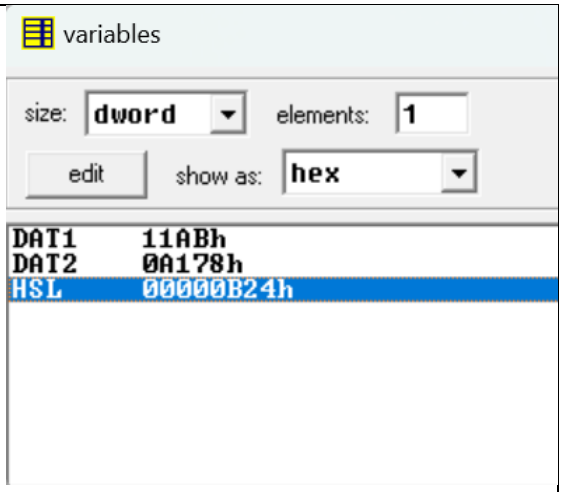
2. Instruksi MUL	Hasil program

```
TITLE SOAL 2 METODE ;MUL
.MODEL SMALL
.STACK 100H
.DATA
    DAT1 DW 11ABH
    DAT2 DW 0A178H
    HSL  DD 0
.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    MOV AX,DAT1
    MUL DAT2

    MOV WORD PTR HSL,DX
    MOV WORD PTR HSL+2,DI

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```



3. Pergeseran SHR

Hasil program

```
TITLE SOAL 2 METODE ;SHR
.MODEL SMALL
.STACK 100H
.DATA
DAT     DW 21A2H
DAT2    DW 12ABH
HSL     DW 0,0
.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS, AX

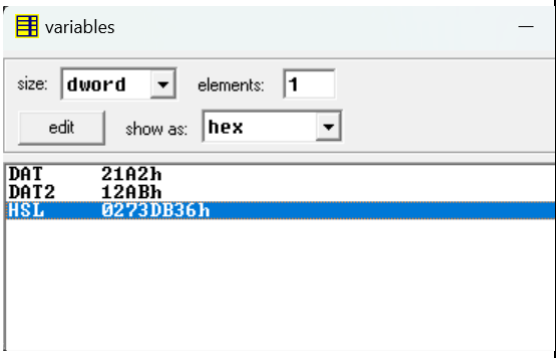
    MOV BX, DAT
    MOV CX, DAT2
    XOR DI,DI
    XOR DX,DX
    MOV AX,16

LUP1:SHR CX,1
     JNC LUP2
     ADD DI,BX
     ADC DX,0

LUP2:SHR DI,1
     RCR DX,1
     DEC AX
     JNZ LUP1

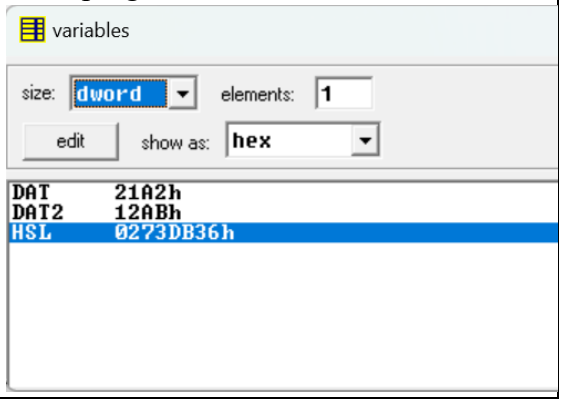
     MOV WORD PTR HSL,DX
     MOV WORD PTR HSL+2,DI

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```



4. Pergeseran SHL

Hasil program



```
01 TITLE SOAL 2 METODE ;SHL
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 DAT      DW 21A2H
06 DAT2     DW 12ABH
07 HSL      DW 0,0
08 .CODE
09 MAIN PROC
10      MOV AX,@DATA
11      MOV DS, AX
12
13      MOV BX, DAT
14      MOV CX, DAT2
15      XOR DX,DX
16      XOR SI,SI
17      MOV AX,16
18
19 LUP1:SHL SI,1
20       SHL DX,1
21       ADC SI,0
22       SHL CX,1
23       JNC LUP2
24       ADD DX,BX
25       ADC SI,0
26
27 LUP2:DEC AX
28       JNZ LUP1
29
30       MOV WORD PTR HSL,DX
31       MOV WORD PTR HSL+2,SI
32
33      MOV AH,4CH
34      INT 21H
35 MAIN ENDP
36 END MAIN
```

Analisis data :

1) Pada program 1 melakukan perkalian dua bilangan heksadesimal menggunakan metode penjumlahan berulang. Nilai DAT1 dimuat ke register BX sebagai bilangan yang akan dijumlahkan, dan DAT2 dimuat ke CX, lalu disalin ke AX sebagai counter jumlah pengulangan. Selama perulangan, nilai BX akan terus ditambahkan ke DX (hasil bawah), dan jika terjadi carry, akan ditambahkan ke DI (hasil atas) menggunakan ADC. Proses ini diulang sebanyak nilai DAT2, hingga hasil akhir berupa bilangan 32-bit tersusun di DX:DI. Setelah selesai, hasil disimpan ke variabel HASIL, dengan DX sebagai nilai rendah dan DI sebagai niali tinggi.
2) Pada program 2 yaitu mengalikan 2 bilangan heksadesimal 5 digit yang kemudian disimpan pada data area. dengan DAT1 sebagai pengali dan DAT 2 sebagai pengalinya. Kedua nilai tersebut disimpan di area data, kemudian DAT1 dipindahkan ke register AX, karena instruksi MUL secara default akan mengalikan isi AX dengan operand yang diberikan (dalam hal ini DAT2). Hasil dari perkalian ini secara otomatis tersimpan di register DX:AX, di mana AX berisi nilai rendah dan DX berisi nilai tinggi dari hasilnya. Selanjutnya, hasil tersebut disimpan ke variabel HSL menggunakan MOV WORD PTR, agar bisa ditampilkan pada variabel.
3) Pada program 3 melakukan perkalian dua bilangan dengan metode penggeseran ke kanan (SHR), yang dikenal juga sebagai metode shift and add. Bilangan pertama (DAT1) dimuat ke register BX, sedangkan bilangan kedua (DAT2) dimuat ke CX sebagai pengontrol loop dan sumber bit LSB.

Proses dilakukan sebanyak 16 kali, di mana setiap bit dari CX dianalisis; jika bit LSB (carry) bernilai 1, maka nilai BX ditambahkan ke DI, dan hasil carry ditambahkan ke DX menggunakan ADC. Setelah penambahan, hasil pada register DI dan DX digeser ke kanan dengan SHR dan RCR, untuk menjaga posisi bit hasil dan carry tetap akurat selama perulangan. Setelah selesai, hasil akhir 32-bit dari perkalian disimpan di variabel HSL, dengan DX sebagai bagian atas dan DI sebagai bagian bawah, lalu program ditutup dengan instruksi INT 21H.

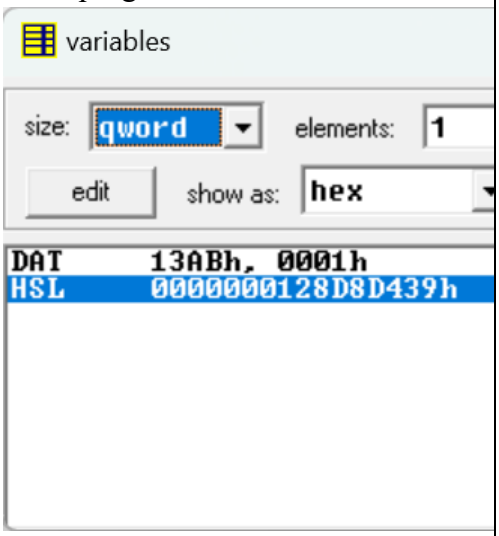
4) Pada program 4 melakukan perkalian dua bilangan heksadesimal menggunakan metode penggeseran ke kiri (SHL). Bilangan pertama (DAT1) dimuat ke register BX sebagai pengali, sedangkan bilangan kedua (DAT2) dimuat ke CX sebagai bilangan yang dikalikan. Perulangan dilakukan sebanyak 16 kali karena operand 16-bit, di mana setiap iterasi akan menggeser bit-bit pada register SI (low word hasil) dan DX (high word hasil) ke kiri untuk mengalikan hasil secara bertahap. Jika bit yang tergeser dari CX menghasilkan carry, maka nilai BX akan ditambahkan ke DX, dan carry dari penjumlahan ditambahkan ke SI menggunakan instruksi ADC. Setelah proses selesai, hasil perkalian 32-bit disimpan ke dalam variabel HASIL di area data, dengan DX sebagai upper 16-bit dan SI sebagai lower 16-bit, lalu program diakhiri dengan INT 21H.

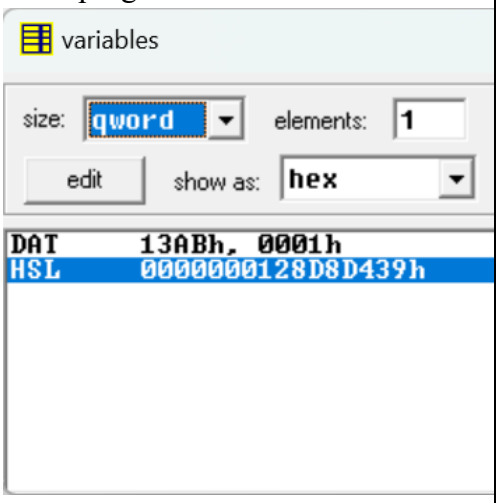
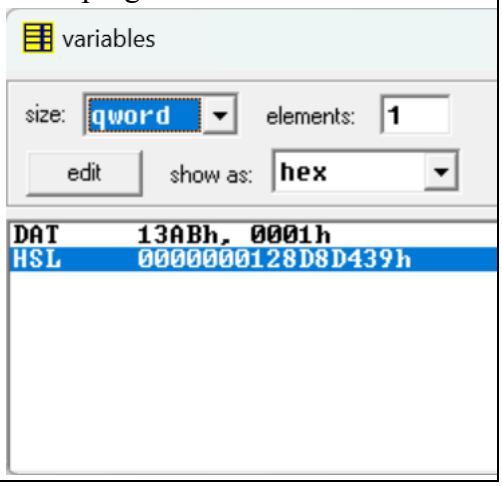
3. Buat program untuk menghitung nilai kuadrat dari data heksadesimal 5 digit, simpa data pada data area dan tampilkan outputnys. Gunakan 4 metode penyelesaian.

1. Penjumlahan berulang

```
01 TITLE SOAL 3 PENJ BERULANG
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 DAT DW 13ABH, 0001H
06 HSL DW 0,0,0
07 CO DW 0,0
08 .CODE
09 MAIN  PROC
10       MOV AX, @DATA
11       MOV DS, AX
12       MOV AX, [DAT]
13       MOV [CO], AX
14       MOV AX, [DAT+2]
15       MOV [CO+2], AX
16
17 L0:   MOV AX, [CO]
18       OR AX, [CO+2]
19       JZ OP
20       MOV AX, [DAT]
21       ADD [HSL], AX
22       MOV AX, [DAT+2]
23       ADC [HSL+2], AX
24       MOV AX, 0
25       ADC [HSL+4], AX
26       MOV AX, [CO]
27       SUB AX, 1
28       MOV [CO], AX
29       MOV AX, [CO+2]
30       SBB AX, 0
31       MOV [CO+2], AX
32       JMP L0
33 OP:
34       MOV AH, 4CH
35       INT 21H
36
37 MAIN ENDP
38 END MAIN
```

Hasil program

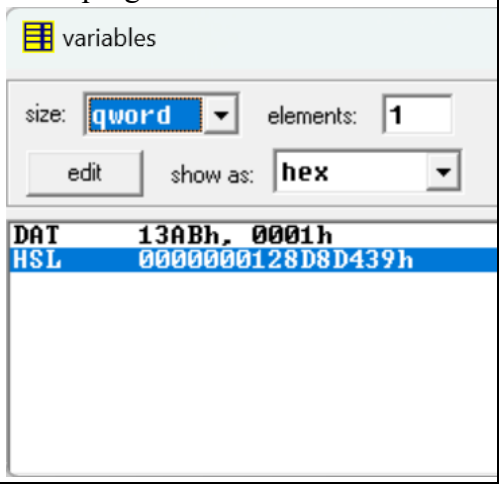


2. Instruksi MUL 01 TITLE KUADRAT DATA HEKSA 5 DIGIT ;METODA MUL 02 .MODEL SMALL 03 .STACK 100H 04 .DATA 05 DAT DW 13ABH, 0001H 06 HSL DW 0,0,0 07 .CODE 08 MAIN PROC 09 MOV AX, @DATA 10 MOV DS, AX 11 12 MOV AX, DAT 13 MUL AX 14 MOV [HSL], AX 15 MOV [HSL+2], DX 16 MOV AX, [DAT+2] 17 MUL DAT 18 MOV BX, AX 19 MOV CX, DX 20 ADD AX, BX 21 ADC DX, CX 22 ADD [HSL+2], AX 23 ADC [HSL+4], DX 24 MOV AX, [DAT+2] 25 MUL AX 26 ADD [HSL+4], AX 27 28 MOV AH, 4CH 29 INT 21H 30 31 MAIN ENDP 32 END MAIN	Hasil program 
3. Pergeseran SHR	Hasil program 

```
01 TITLE SOAL 3 KUADRAT ;SHR
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 DAT    DD 113ABH
06 HSL    DD ?
07 KUD    DD ?
08 .CODE
09 MAIN PROC
10       MOV AX,@DATA
11       MOV DS,AX
12
13       MOV SI,WORD PTR DAT
14       MOV DI,WORD PTR DAT+2
15
16       MOV BX,WORD PTR DAT
17       MOV CX,WORD PTR DAT+2
18
19       XOR AX,AX
20       XOR DX,DX
21       MOV BP,32
22
23 LUP:  TEST BX,1
24       JZ LIP
25       ADD AX,SI
26       ADC DX,DI
27
28 LIP:  SHL SI,1
29       RCL DI,1
30       SHR CX,1
31       RCR BX,1
32       DEC BP
33       JNZ LUP
34
35
36       MOV WORD PTR HSL, AX
37       MOV WORD PTR HSL+2,DX
38
39       MOV AH,4CH
40       INT 21H
41 MAIN ENDP
42       END MAIN
```

4. Pergeseran SHL

Hasil program



```
01 TITLE SOAL 3 KUADRAT;SHL
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 DAT    DD 113ABH
06 HASIL  DD ?
07 KUD    DD ?
08 .CODE
09 MAIN PROC
10       MOV AX,@DATA
11       MOV DS,AX
12
13       MOV BX,WORD PTR DAT
14       MOV CX,WORD PTR DAT+2
15
16       MOV SI,WORD PTR DAT
17       MOV DI,WORD PTR DAT+2
18
19       XOR AX,AX
20       XOR DX,DX
21       MOV BP,32
22
23 LUP:
24       SHL AX,1
25       RCL DX,1
26       SHL BX,1
27       RCL CX,1
28       JNC LIP
29
30       ADD AX,WORD PTR DAT
31       ADC DX,WORD PTR DAT+2
32
33 LIP:
34       DEC BP
35       JNZ LUP
36
37       MOV WORD PTR HASIL, AX
38       MOV WORD PTR HASIL+2,DX
39
40       MOV AH,4CH
41       INT 21H
42 MAIN ENDP
43       END MAIN
```

Analisis Data :

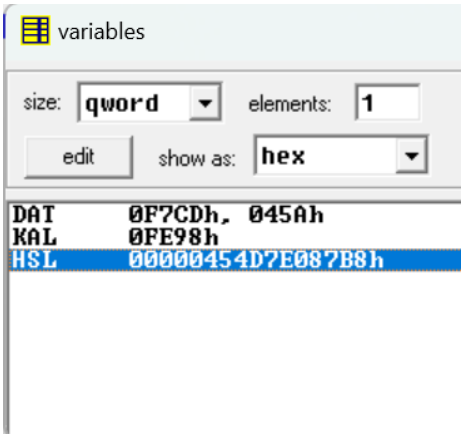
1) Pada program 1 adalah penjumlahan berulang di mana dua bilangan dijumlahkan berkali-kali sesuai dengan bit aktif dari angka kedua. Untuk mengkuadratkan dapat menggunakan metode penjumlahan berulang dengan menambahkan angka pertama (misalnya, 113ABH) ke dirinya sendiri beberapa kali berdasarkan bit yang ada pada angka kedua.
2) Pada program 2 menggunakan instruksi MUL. Instruksi ini akan mengalikan dua bilangan. Untuk menghitung kuadrat 113ABH, maka nilai 113ABH dikali dengan dirinya sendiri. Program akan memindahkan angka 113ABH ke dalam register AX, kemudian menggunakan instruksi MUL AX untuk mengalikan angka tersebut dengan dirinya sendiri. Hasilnya akan disimpan di data area.
3) Pada program 3 dilakukan pergeseran bit ke kiri (SHL) untuk menggandakan nilai. Misalnya, untuk mengkuadratkan 113ABH, maka angka akan digeser ke kiri secara bertahap dan menjumlahkan hasilnya. Setiap kali angka digeser ke kiri, nilai tersebut akan bertambah dua kali lipat (karena pergeseran bit ke kiri setara dengan perkalian). Kemudian, menggeser angka dua kali dan menambahkan hasil pergeseran untuk menghitung kuadratnya. Begitupun program nomor 4 yang tidak jauh berbeda, hanya saja, SHR akan menggeser bit ke kanan secara bertahap.

4. Buat program untuk mengalikan 2 data heksadesimal berikut : 45AF7CD * FE98.Pergunakan 2 cara penyelesaian.

1. Penjumlahan berulang

```
01  TITLE SOAL 4 ;PENJ BERULANG
02  .DATA
03  DAT DW 0F7CDH, 045AH
04  KAL DW 0FE98H
05  HSL DW 0,0,0
06  .CODE
07  MAIN  PROC
08        MOV AX, @DATA
09        MOV DS, AX
10
11        MOV CX, KAL
12  L1:   CMP CX, 0
13        JE PO
14        MOV AX, DAT
15        ADD HSL, AX
16        MOV AX, DAT+2
17        ADC HSL+2, AX
18        MOV AX, 0
19        ADC HSL+4, AX
20        DEC CX
21        JMP L1
22  PO:
23        MOV AH, 4CH
24        INT 21H
25
26
27  MAIN ENDP
28  END MAIN
```

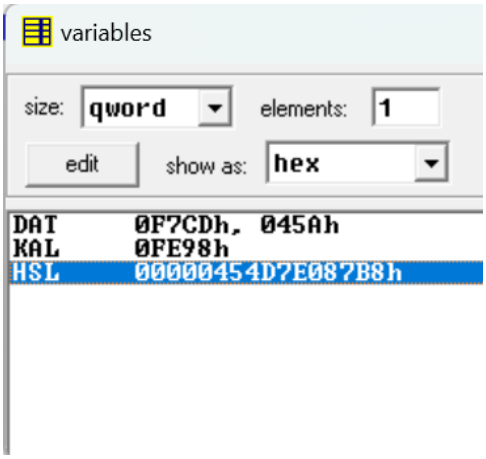
Hasil Program



2. Instruksi MUL

```
01  TITLE SOAL 4; MUL
02  .MODEL SMALL
03  .STACK 100H
04  .DATA
05  DAT DW 0F7CDH, 045AH
06  KAL DW 0FE98H
07  HSL DW 0,0,0
08  .CODE
09  MAIN  PROC
10        MOV AX, @DATA
11        MOV DS, AX
12        MOV AX, DAT
13        MUL KAL
14        MOV [HSL], AX
15        MOV [HSL+2], DX
16
17        MOV AX, [DAT+2]
18        MUL KAL
19        ADD [HSL+2], AX
20        ADC [HSL+4], DX
21
22        MOV AH, 4CH
23        INT 21H
24  MAIN ENDP
25  END MAIN
```

Hasil Program



F. KESIMPULAN

Dari hasil praktikum dan percobaan dengan menyelesaikan empat soal diatas, dapat disimpulkan bahwa membuktikan bahwa metode perkalian dalam Assembly dapat dilakukan dengan berbagai meotde yaitu penjumlahan berulang, instruksi MUL(multiplication), pergeseran ke kanan (SHR) dan pergeseran ke kiri (SHL). Instruksi MUL merupakan metode yang paling efisien karena langsung menghitung hasil dalam satu langkah, menghasilkan nilai 32-bit dari perkalian dua register 16-bit (AX × reg/mem). Sementara itu, metode SHL (shift left) dan SHR (shift right) merupakan teknik logika bit yang berbasis biner, dan membutuhkan kontrol tambahan untuk mengelola carry serta hasil akhir. Metode pergeseran ini bekerja secara berulang dalam loop, dan digunakan untuk membentuk hasil secara bertahap. Kemudian, metode penjumlahan berulang adalah cara paling dasar dan sederhana, di mana bilangan pertama sebagai yang dikali dijumlahkan terus-menerus sebanyak nilai bilangan kedua sebagai pengali. Namun, metode ini akan memerlukan banyak iterasi jika pengali bernilai besar. Sehingga pada praktikum ini, praktikan dapat membuat program dan memahami metode mana yang lebih efisien untuk melakukan operasi perkalian pada bahasa assembly yang dijalankan pada EMU8086.

DAFTAR PUSTAKA

A. E. Putra, "Belajar bahasa assembly dengan EMU 8086," [Online]. Available: https://digilib.amikompurwokerto.ac.id/index.php?p=show_detail&id=13246.

Buggs, "MICROPROCESSOR EMULATOR," 2018. [Online]. Available: https://emu8086-microprocessor-emulator.en.softonic.com/. [Accessed 1 April 2025].

Hasanah, Rahmawati. 2019. Mikroprosesor dan Interface. Bandung: Politeknik Negeri Bandung

S.to. 2001. “Pemrograman Dengan Bahasa Assembly Edisi Online Versi 1.0